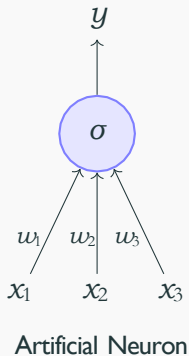


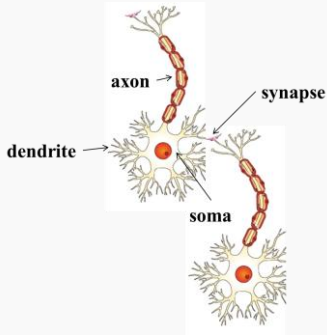
# Biological Neurons



- The most fundamental unit of a deep neural network is called an *artificial neuron*
- Why is it called a neuron ? Where does the inspiration come from ?
- The inspiration comes from biology (more specifically, from the *brain*)
- *biological neurons = neural cells = neural processing units*

*OR*

An **artificial neuron** is a mathematical model **inspired by the biological neuron** in the human brain.



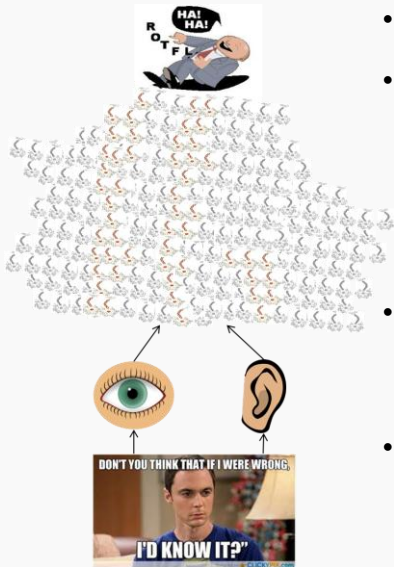
Biological Neurons\*

- **Dendrite:** Receives signals from other neurons.
- **Synapse:** The point of connection between neurons.
- **Soma (Cell body):** Processes the information in the neuron.
- **Axon:** Transmits the output of the neuron to other neurons or cells.

---

\*Image adapted from

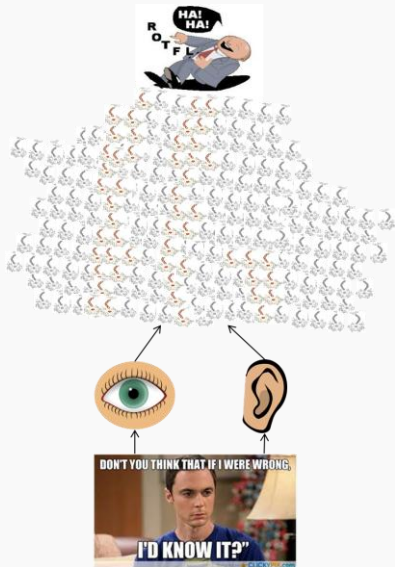
<https://cdn.vectorstock.com/i/composite/12,25/neuron-cell-vector-81225.jpg>



- In reality, it is not just one neuron that does all the work.
- The brain has a very large network of many neurons connected to each other and working at the same time (massively parallel network).

Sense organs (**eyes, ears, skin, etc.**) send information to the **first/lowest layer** of neurons in the brain.

- Some of these neurons may become active (fire) (**in red**) when they get this information and then pass it to other connected neurons.
- Those neurons may also fire (**again, in red**) and pass the message forward. **This process continues until it finally produces a response** (laughter in this case).
- An average **human brain has** about  **$10^{11}$  neurons**, which is around 100 billion neurons!



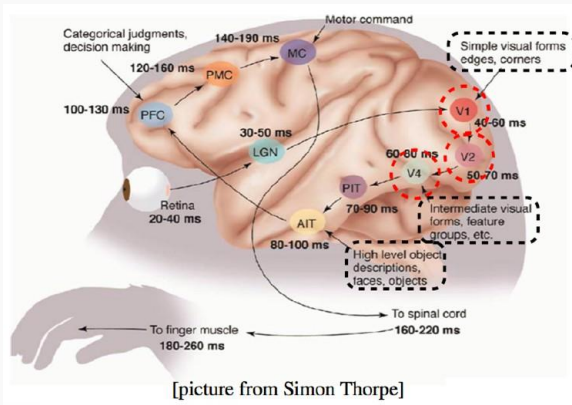
- This massively parallel network also ensures that there is division of work
- Each neuron may perform a certain role or respond to a certain stimulus

*fires if at least  
2 of the 3 inputs fired*

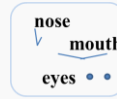


A simplified illustration

- This massively parallel network also ensures that there is division of work
- Each neuron may perform a certain role or respond to a certain stimulus



**Layer 1: detect edges & corners**



**Layer 2: form feature groups**



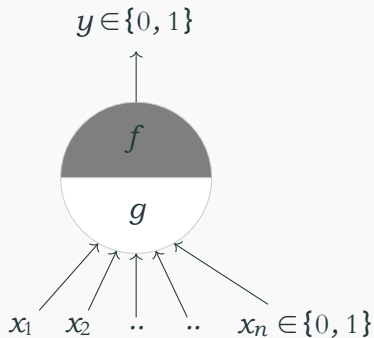
**Layer 3: detect high level objects, faces, etc.**

Sample illustration of hierarchical processing\*

\*Idea borrowed from Hugo Larochelle's lecture slides

## **McCulloch Pitts Neuron**





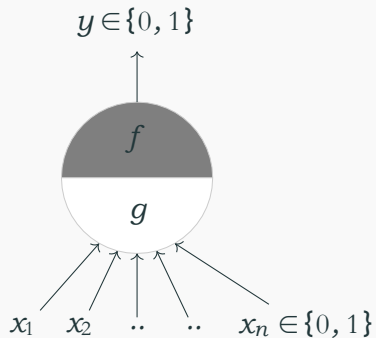
- In 1943, McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (Earliest ANN model).
- **g** aggregates the inputs and the function **f** takes a decision based on this aggregation
- The inputs can be excitatory or inhibitory  
 $y = 0$  if any  $x_i$  is inhibitory, else

$$g(x_1, x_2, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = 1 \quad \text{if} \quad g(\mathbf{x}) \geq \theta$$

$$= 0 \quad \text{if} \quad g(\mathbf{x}) < \theta$$

- $\theta$  is called the **thresholding parameter**



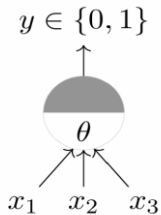
**Inhibitory input:** This input **decreases** the likelihood that a neuron will fire.

**Example:** If I am ill, I cannot go to watch movie. So, it does not matter what other inputs are or factors are

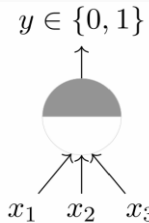
**Excitatory Input:** This input **increases** the likelihood that a neuron will activate (fire).

Implementing some boolean functions using McCulloch Pitts (MP) neuron

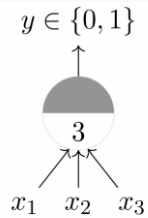
# AND Function



A McCulloch Pitts unit

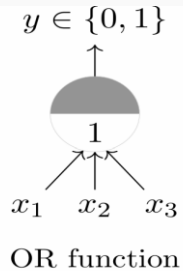
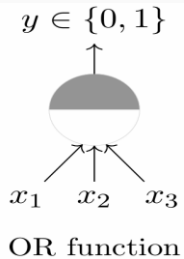


AND function

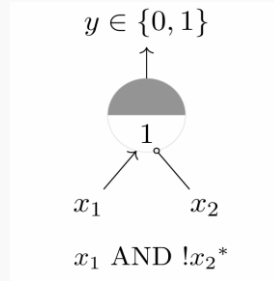
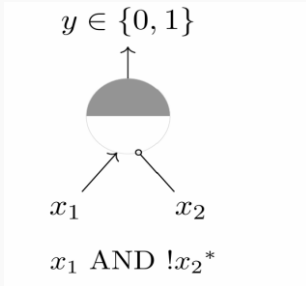


AND function

# OR Function

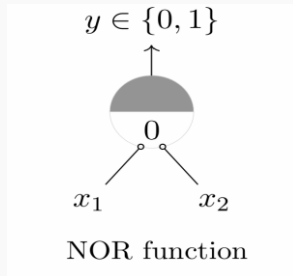
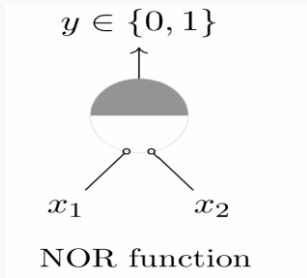


## $x_1$ AND $\neg x_2$

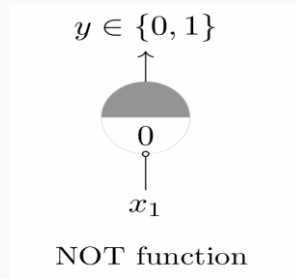
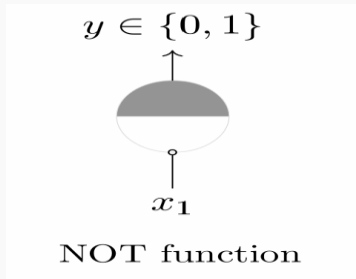


Note: Circle at the end indicates inhibitory input: if any inhibitory input is 1 the output will be 0

# NOR Function



# NOT Function

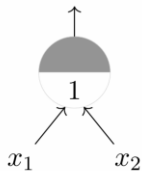




- Can any boolean function be represented using a McCulloch Pitts unit ?

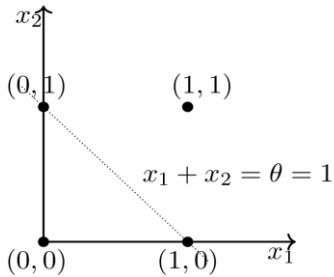
## Geometric interpretation of a MP unit: OR

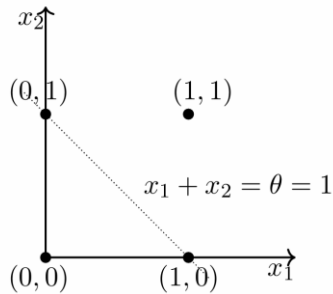
$$y \in \{0, 1\}$$



OR function

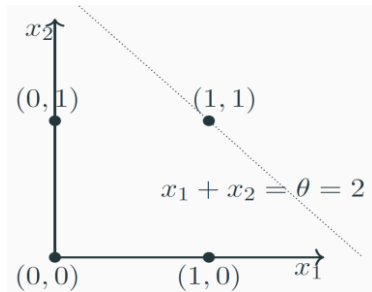
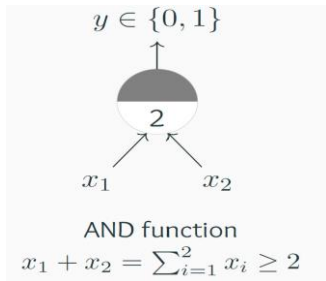
$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 1$$

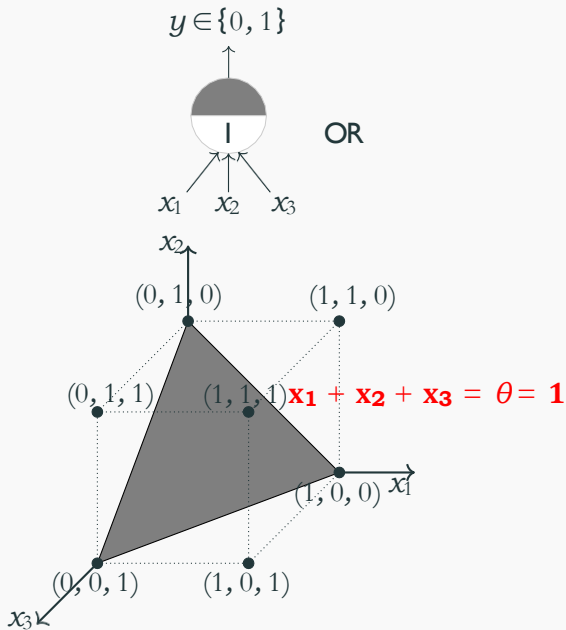




- A single MP neuron splits the input points (4 points for 2 binary inputs) into two halves
- Points lying on or above the line  $\sum_{i=1}^n x_i - \theta = 0$  and points lying below this line
- In other words, all inputs which produce an output 0 will be on one side ( $\sum_{i=1}^n x_i < \theta$ ) of the line and all inputs which produce an output 1 will lie on the other side ( $\sum_{i=1}^n x_i \geq \theta$ ) of this line

## Geometric interpretation of a MP unit: AND





- What if we have more than 2 inputs?
- Instead of a line we will have a plane
- For the OR function, we want a plane such that the point  $(0,0,0)$  lies on one side and the remaining 7 points lie on the other side of the plane

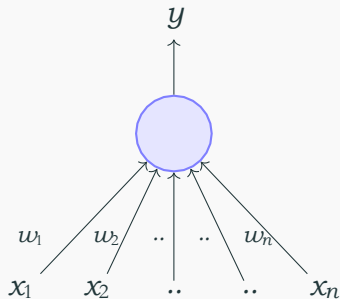
- A single McCulloch Pitts Neuron can be used to represent boolean functions which are linearly separable
- Linear separability (for boolean functions) : There exists a line (plane) such that all inputs which produce a 1 lie on one side of the line (plane) and all inputs which produce a 0 lie on other side of the line (plane)

# Perceptron

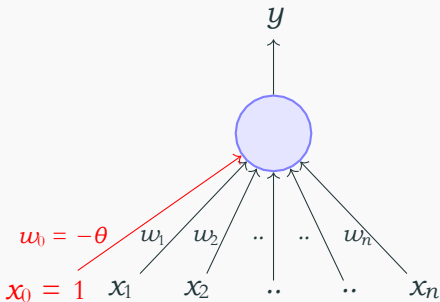
## The story ahead ...

- What about non-boolean (say, real) inputs ?
- Do we always need to hand code the threshold ?
- Are all inputs equal ? What if we want to assign more weight (importance) to some inputs ?
- What about functions which are not linearly separable ?





- In 1958, Frank Rosenblatt, an American psychologist, proposed the **classical perceptron model**.
- The perceptron is a **more general computational model** than the **McCulloch–Pitts neuron** model.
- The main differences are:
  - Introduction of **numerical weights** for inputs.
  - A **learning mechanism** to adjust these weights.
- Inputs are no longer limited to boolean values, i.e.; they can be real numbers.
- The perceptron model was refined and carefully analyzed by Marvin Minsky and Seymour Papert in 1969. In their work, this model is commonly referred to as **the perceptron model**.



$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$

$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

A more accepted convention,

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

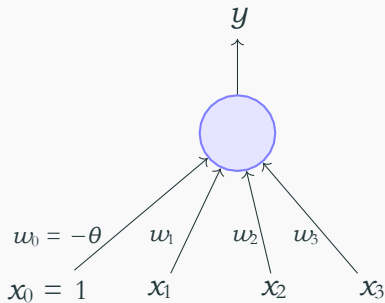
where,  $x_0 = 1$  and  $w_0 = -\theta$

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

We will now try to answer the following questions:

- Why are we trying to implement boolean functions?
- Why do we need weights ?
- Why is  $w_0 = -\theta$  called the bias ?

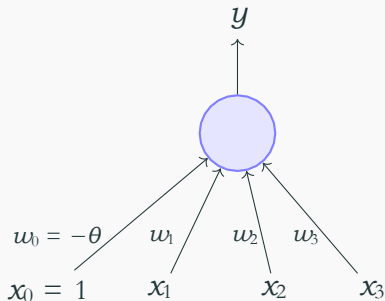


$$x_1 = \textit{isActorDamon}$$

$$x_2 = \textit{isGenreThriller}$$

$$x_3 = \textit{isDirectorNolan}$$

- Consider the task of predicting whether we would like a movie or not
- Suppose, we base our decision on 3 inputs (binary, for simplicity)
- Based on our past viewing experience (**data**), we may give a high weight to *isDirectorNolan* as compared to the other inputs
- Specifically, even if the actor is not *Matt Damon* and the genre is not *thriller* we would still want to cross the threshold  $\theta$  by assigning a high weight to *isDirectorNolan*



$x_1 = isActorDamon$

$x_2 = isGenreThriller$

$x_3 = isDirectorNolan$

- $w_0$  is called the bias as it represents the prior (prejudice) of the model
- A movie buff may have a very low threshold and may watch any movie irrespective of the genre, actor, director [ $\theta = 0$ ]
- On the other hand, a selective viewer may only watch thrillers starring Matt Damon and directed by Nolan [ $\theta = 3$ ], if be modeled using an AND-like decision rule
- The weights ( $w_1, w_2, \dots, w_n$ ) and the bias ( $w_0$ ) will depend on the data (viewer history in this case)

What kind of functions can be implemented using the perceptron? Any difference from McCulloch Pitts neurons?

## McCulloch Pitts Neuron

(assuming no inhibitory inputs)

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n x_i < 0$$

## Perceptron

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

- From the equations it should be clear that even a perceptron separates the input space into two halves
- All inputs which produce a 1 lie on one side and all inputs which produce a 0 lie on the other side
- In other words, a single perceptron can only be used to implement linearly separable functions
- Then what is the difference? **The weights (including threshold) can be learned and the inputs can be real valued**
- We will first revisit some boolean functions and then see the perceptron learning algorithm (for learning weights)

| $x_1$ | $x_2$ | OR |                                     |
|-------|-------|----|-------------------------------------|
| 0     | 0     | 0  | $w_0 + \sum_{i=1}^2 w_i x_i < 0$    |
| 1     | 0     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 0     | 1     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |
| 0     | 1     | 1  | $w_0 + \sum_{i=1}^2 w_i x_i \geq 0$ |

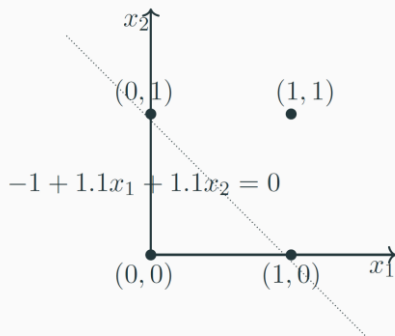
$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 \geq -w_0$$

One possible solution to this set of inequalities is  $w_0 = -1, w_1 = 1.1, w_2 = 1.1$  (and various other solutions are possible)



Note that we can come up with a similar set of inequalities and find the value of  $\theta$  for a McCulloch Pitts neuron also

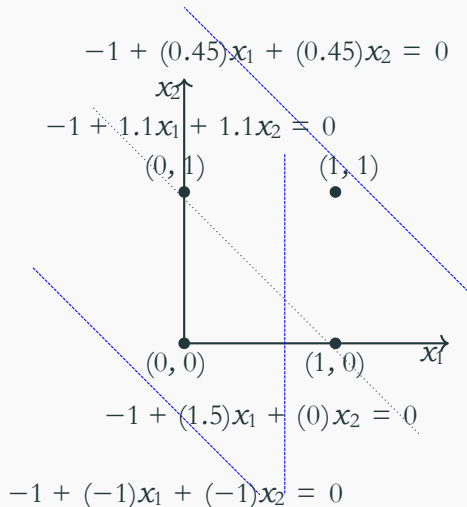


## **Errors and Error Surfaces**

- Let us fix the threshold ( $-w_0 = 1$ ) and try different values of  $w_1, w_2$
- Say,  $w_1 = -1, w_2 = -1$
- What is wrong with this line? We make an error on 1 out of the 4 inputs
- Lets try some more values of  $w_1, w_2$  and note how many errors we make

| $w_1$ | $w_2$ | errors |
|-------|-------|--------|
| -1    | -1    | 1      |
| 1.5   | 0     | 1      |
| 0.45  | 0.45  | 3      |

- We are interested in those values of  $w_0, w_1, w_2$  which result in 0 error



- For ease of analysis, we will keep  $w_0$  fixed (-1) and try the error for different values of  $w_1$ ,  $w_2$
- For a given  $w_0$ ,  $w_1$ ,  $w_2$  we will compute  $-w_0 + w_1 * x_1 + w_2 * x_2$  for all combinations of  $(x_1, x_2)$  and note down how many errors we make
- For the OR function, an error occurs if  $(x_1, x_2) = (0, 0)$  but  $-w_0 + w_1 * x_1 + w_2 * x_2 \geq 0$  or if  $(x_1, x_2) \neq (0, 0)$  but  $-w_0 + w_1 * x_1 + w_2 * x_2 < 0$
- **Perceptron Learning Algorithm** be used to learning these weights and threshold (Watch NPTEL Video)

## Questions

- What about non-boolean (say, real) inputs? **Real valued inputs are allowed in perceptron**
- Do we always need to hand code the threshold? **No, we can learn the threshold**
- Are all inputs equal? What if we want to assign more weight (importance) to some inputs? **A perceptron allows weights to be assigned to inputs**
- What about functions which are not linearly separable ? **Not possible with a single perceptron**

## Theorem

Any boolean function of  $n$  inputs can be represented exactly by a network of perceptrons containing **1 hidden layer** with  $2^n$  perceptrons and **one output layer** containing 1 perceptron. (This theorem gives **the representation power of a MLP** )

**Proof (informal:)** See NPTEL Vedio

**Note:** A network of  $2^n + 1$  perceptrons is not necessary but sufficient. For example, we already saw how to represent AND function with just 1 perceptron

**Catch:** As  $n$  increases the **number of perceptrons in the hidden layers** increases exponentially

## Note

- Networks of the form containing, **an input, output and one or more hidden layers** are called **Multilayer Perceptrons** (MLP, in short)
- More appropriate terminology would be “**Multilayered Network of Perceptrons**” but MLP is the more commonly used name
- A MLP with a single hidden layer can represent **any** boolean function

## Note

Each perceptron in the hidden layer is designed to **detect one specific input combination**

For 3 inputs, there are **8 unique combinations** So, we use **8 perceptrons**, one for each combination

Each hidden neuron acts like a **pattern detector**